

HopfNet-MHD: Progress

Milestones 1–4 — Spectral Infrastructure, Gauge Projection,
Hopf Link Initialization, and the Nonlinear RHS Pipeline

Mohana Rangan Designan

June 18, 2026

Abstract

This report documents the implementation and validation of the first four milestones of the HopfNet-MHD Phase 1 engine: a 128^3 pseudo-spectral incompressible Hall-magnetohydrodynamics (Hall-MHD) solver targeting Apple Silicon (M4, 16 GB unified memory). We report on (1) the spectral grid, wavevector arrays, real-to-complex FFT wrappers, and Orszag 2/3 dealiasing mask; (2) the Coulomb-gauge orthogonal projection tensor $\mathcal{P}(\mathbf{k})$ and the spectral curl operator; (3) the regularized Biot–Savart construction of the magnetic Hopf link initial condition; and (4) the fully dealiased, gauge-consistent nonlinear right-hand-side (RHS) pipeline for both the induction equation and an incompressible Navier–Stokes velocity field, coupled through the Lorentz force $\mathbf{J} \times \mathbf{B}$. Each milestone is accompanied by a battery of unit tests verifying the relevant analytic identities to machine precision on an $N = 8$ test grid. All derivations reference the governing equations established in the companion document *Mathematical Foundations of the HopfNet-MHD Engine* (Designan, 2026).

Contents

1	Architecture Overview	2
2	Milestone 1: Spectral Grid and FFT Infrastructure	2
2.1	Mathematical Basis	2
2.2	Implementation	2
2.3	Validation	3
3	Milestone 2: Coulomb Gauge Projection and Spectral Curl	3
3.1	Mathematical Basis	3
3.2	Implementation	3
3.3	Validation	4
4	Milestone 3: Magnetic Hopf Link Initial Condition	4
4.1	Mathematical Basis	4
4.2	Implementation	4
4.3	Validation	5
4.4	Note on the 128^3 Production Run	5
5	Milestone 4: The Nonlinear RHS Pipeline	5
5.1	Mathematical Basis	5
5.2	Implementation	6
5.3	Validation	6
6	Summary of Test Coverage	7

7	Current Status and Next Steps	7
7.1	What Is Validated	7
7.2	What Remains for Phase 1	7

1 Architecture Overview

The engine follows a hybrid C++/Python architecture, chosen to balance numerical performance against development velocity on a single 16 GB Apple Silicon workstation.

- **C++ core** (`src/`, compiled via CMake + pybind11 into the extension module `hopfnet_cpp`): owns the performance- and precision-critical primitives — the spectral grid, the FFTW3-backed 3D real-to-complex transform pair, the Coulomb gauge projection tensor, the spectral curl operator, and the regularized Biot–Savart Hopf link initializer.
- **Python orchestration layer** (`python/hopfnet/rhs.py`): assembles the nonlinear RHS for both evolved fields ($\mathbf{A}$ and $\mathbf{v}$) by composing the C++ primitives. Every floating-point-heavy operation (FFTs, projections, curls) executes inside compiled code; the Python layer performs only array bookkeeping via NumPy, which itself dispatches to C.
- **Test suite** (`tests/`): one file per milestone, run with `pytest`, executed on a small $N = 8$ grid where analytic ground truth is tractable by hand.

This split keeps the stiff time-integration loop (Milestone 5–6) free to be implemented entirely in C++ once the underlying operators are validated, while allowing the nonlinear-term bookkeeping to be developed and debugged interactively in Python first.

2 Milestone 1: Spectral Grid and FFT Infrastructure

2.1 Mathematical Basis

The pseudo-spectral method represents all fields as truncated Fourier series (Sec. 2.1 of Designan, 2026, Eq. 10),

$$\mathbf{A}(\mathbf{x}, t) = \sum_{\mathbf{k}} \hat{\mathbf{A}}(\mathbf{k}, t) e^{i\mathbf{k} \cdot \mathbf{x}}, \quad (1)$$

on a triply periodic domain of side $L = 2\pi$. For a real-valued field sampled on N^3 points, the discrete wavevector components take the standard FFT ordering $n \in \{0, 1, \dots, N/2, -(N/2 - 1), \dots, -1\}$, with $k_i = (2\pi/L) n_i$.

2.2 Implementation

`src/spectral_grid.hpp` implements the class `SpectralGrid`, which precomputes and stores:

- 1D wavenumber arrays $k_x, k_y \in \mathbb{R}^N$ and $k_z \in \mathbb{R}^{N/2+1}$ (the last axis is truncated under the real-to-complex, or rFFT, layout, exploiting the Hermitian symmetry $\hat{\mathbf{A}}(-\mathbf{k}) = \hat{\mathbf{A}}^*(\mathbf{k})$ of a real field — Sec. 2.4/§3 of Designan, 2026);
- the flattened array $|\mathbf{k}|^2 \in \mathbb{R}^{N \times N \times (N/2+1)}$;
- the Orszag 2/3 dealiasing mask: a binary array that is 1 where $|n_x|, |n_y|, |n_z| \leq \lfloor (2/3)(N/2) \rfloor$ and 0 otherwise.

`src/fft3d.hpp` implements `FFT3D`, a thin RAII wrapper around a single pair of FFTW3 `r2c/c2r` plans created with `FFTW_MEASURE`. The forward transform returns the unnormalized FFTW output; the inverse transform divides by N^3 , so that `inverse(forward(f)) = f` to machine precision. Both methods are exposed to Python via pybind11 with NumPy array marshalling.

2.3 Validation

Six unit tests in `tests/test_milestone1.py` establish:

1. **Round-trip accuracy:** for a random field and for a constant field, `inverse(forward(f))` reproduces f to 10^{-12} .
2. **Normalization:** a constant field $f \equiv 3.7$ transforms to a single nonzero coefficient at $\mathbf{k} = 0$ equal to $3.7N^3$, confirming the $1/N^3$ inverse normalization.
3. **Single-mode spectral purity:** $\cos(x)$ (integer wavenumber $n_x = \pm 1$) places $> 99\%$ of its spectral energy at the expected indices 1 and $N - 1$.
4. **Wavevector correctness:** for $N = 8$, the computed k_x, k_y, k_z arrays match the hand-derived FFT ordering $\{0, 1, 2, 3, 4, -3, -2, -1\} \times (2\pi/L)$ and $\{0, 1, 2, 3, 4\} \times (2\pi/L)$ respectively.
5. $|\mathbf{k}|^2$ **consistency:** spot-checked against direct reconstruction from k_x, k_y, k_z .
6. **Dealias mask correctness:** for $N = 8$ the cutoff is $\lfloor (2/3) \cdot 4 \rfloor = 2$; modes with $|n| \leq 2$ are retained, $|n| \geq 3$ (including the Nyquist mode) are removed, and the mask is symmetric under $n \rightarrow -n$.

All six tests pass.

3 Milestone 2: Coulomb Gauge Projection and Spectral Curl

3.1 Mathematical Basis

As derived in Sec. 2.3 of Desigan, 2026, enforcing $\nabla \cdot \mathbf{A} = 0$ for all time reduces, in Fourier space, to applying the orthogonal projection tensor

$$\mathcal{P}_{ij}(\mathbf{k}) = \delta_{ij} - \frac{k_i k_j}{|\mathbf{k}|^2}, \quad \frac{\partial \hat{\mathbf{A}}}{\partial t} = \mathcal{P}(\mathbf{k}) \hat{\mathbf{R}}. \quad (2)$$

At $\mathbf{k} = 0$ this expression is $0/0$; following Sec. 2.3's resolution, $\mathcal{P}(\mathbf{0}) \equiv \mathbb{I}$, since the spatially uniform mean field trivially satisfies the gauge condition.

The spectral curl follows from $\nabla \rightarrow i\mathbf{k}$ in Fourier space:

$$\hat{\mathbf{B}} = i\mathbf{k} \times \hat{\mathbf{A}}, \quad B_x = i(k_y A_z - k_z A_y), \quad B_y = i(k_z A_x - k_x A_z), \quad B_z = i(k_x A_y - k_y A_x). \quad (3)$$

3.2 Implementation

`src/projection.hpp` implements:

- `project_field(grid, Ax, Ay, Az)`: applies Eq. (2) in place, with an explicit early-exit at $\mathbf{k} = 0$ (identified by $|\mathbf{k}|^2 < 10^{-14}$) implementing $\mathcal{P}(\mathbf{0}) = \mathbb{I}$.
- `spectral_curl(grid, Ax, Ay, Az, Bx, By, Bz)`: implements Eq. (3) component-wise, using the identity $i(a + ib) = -b + ia$ for the multiplication by i .

Both operate on `std::vector<std::complex<double>>` buffers of length $N \times N \times (N/2 + 1)$ and are exposed to Python returning NumPy `complex128` arrays of shape $(N, N, N/2 + 1)$.

3.3 Validation

Five unit tests in `tests/test_milestone2.py`:

1. **Gauge enforcement:** for a random complex vector field $\hat{\mathbf{R}}$, $\mathbf{k} \cdot \mathcal{P}(\mathbf{k})\hat{\mathbf{R}} = 0$ to 10^{-12} for every $\mathbf{k} \neq 0$.
2. **Idempotence:** $\mathcal{P}(\mathcal{P}(\hat{\mathbf{R}})) = \mathcal{P}(\hat{\mathbf{R}})$, confirming $\mathcal{P}^2 = \mathcal{P}$ as required of a projection operator.
3. **$\mathbf{k} = 0$ identity:** the $\mathbf{k} = 0$ mode of $\hat{\mathbf{R}}$ is passed through unchanged.
4. **Curl correctness:** for the analytic field $\mathbf{A} = (0, 0, \sin x)$, the spectral curl (via forward FFT $\rightarrow$ `spectral_curl` $\rightarrow$ inverse FFT) reproduces $\mathbf{B} = (0, -\cos x, 0)$ to 10^{-12} .
5. **Automatic solenoidality:** for a random field, the curl output satisfies $\mathbf{k} \cdot \hat{\mathbf{B}} = 0$ to 10^{-10} , confirming $\nabla \cdot (\nabla \times \mathbf{A}) \equiv 0$ holds at the discrete level.

All five tests pass.

4 Milestone 3: Magnetic Hopf Link Initial Condition

4.1 Mathematical Basis

The initial vector potential is constructed from two regularized Biot–Savart current loops (Sec. 3 of Designan, 2026): Ring 1 in the xy -plane centered at $(0, d, 0)$, Ring 2 in the yz -plane centered at $(0, -d, 0)$, both of radius R . The infinite-wire singularity in the Biot–Savart kernel is regularized by a finite core radius a (Eq. 21),

$$\mathbf{A}_{\text{reg}}(\mathbf{r}) = \frac{\mu_0 I}{4\pi} \oint_C \frac{d\mathbf{l}'}{\sqrt{|\mathbf{r} - \mathbf{r}'|^2 + a^2}}, \quad (4)$$

giving the explicit component integrals of Eqs. 26–28, each a smooth 2π -periodic integral over the loop parameter.

4.2 Implementation

`src/hopf_link.hpp` implements `hopf_link::compute`, which:

- evaluates each of the three component integrals (Eqs. 26–28) at every grid point using composite Simpson’s rule on $[0, 2\pi]$ with an even number of subintervals `n_quad`;
- places the grid on the centered domain $x_i = -L/2 + i(L/N)$, so the two interlocking rings sit symmetrically about the box center;
- returns three real $N \times N \times N$ arrays (A_x, A_y, A_z) .

The Coulomb-gauge projection (Milestone 2) is applied *after* construction, by transforming to Fourier space, calling `project_field`, and (if needed) transforming back — exactly as prescribed in Sec. 3.4 of Designan, 2026.

4.3 Validation

Four unit tests in `tests/test_milestone3.py` (run at $N = 8$, $R = 1$, $d = 0.3$, $a = 0.2$):

1. **Finiteness and non-triviality:** all three components are finite everywhere (no NaN/Inf from the regularized $1/D$ kernels) and are not identically zero.
2. **Quadrature convergence:** the maximum pointwise change in A_x when doubling `n_quad` from 64 to 128 is more than a factor of 2 smaller than the change from 32 to 64 (Richardson-type convergence), and the $64 \rightarrow 128$ change is below 10^{-3} relative to the field’s own magnitude — consistent with Simpson’s rule on a smooth, regularized integrand.
3. **Coulomb gauge after projection:** after `FFT` $\rightarrow$ `project_field`, $\mathbf{k} \cdot \hat{\mathbf{A}} = 0$ to 10^{-12} for all $\mathbf{k} \neq 0$, exactly as required for the initial state to satisfy $\nabla \cdot \mathbf{A} = 0$ to machine precision.
4. **Physical content:** the resulting field $\mathbf{B} = \nabla \times \mathbf{A}$ carries nonzero magnetic energy $\sum |\hat{\mathbf{B}}|^2 > 0$ (the Hopf link is not a trivial/null configuration) and is itself solenoidal, $\mathbf{k} \cdot \hat{\mathbf{B}} = 0$ to 10^{-10} .

All four tests pass.

4.4 Note on the 128^3 Production Run

The quadrature is $\mathcal{O}(N^3 \cdot n_{\text{quad}})$. At $N = 128$ with $n_{\text{quad}} = 64$, this is approximately 1.3×10^8 evaluations of the regularized kernel per Cartesian component — a one-time initialization cost expected to take on the order of tens of seconds on the M4, well within the 16 GB memory budget (~ 260 MB per field).

5 Milestone 4: The Nonlinear RHS Pipeline

This milestone implements the right-hand side of both evolved equations prior to the ETD split (Milestone 5). Two physical fields are evolved: the vector potential $\mathbf{A}$ (Hall-MHD induction, Eq. 8) and an incompressible velocity field $\mathbf{v}$ (momentum balance), coupled through the Lorentz force $\mathbf{J} \times \mathbf{B}$.

5.1 Mathematical Basis

Induction RHS. From Eq. 12 of Desigan, 2026, the raw forcing for $\mathbf{A}$ is

$$\mathbf{R}_A = \mathbf{v} \times \mathbf{B} - \frac{d_i}{\rho}(\mathbf{J} \times \mathbf{B}) - \eta \mathbf{J}, \quad (5)$$

which, after dealiasing, is projected via Eq. (2) to remove its longitudinal component, preserving $\nabla \cdot \mathbf{A} = 0$.

Momentum RHS. For an incompressible velocity field, the nonlinear momentum forcing (advection plus Lorentz force, with the pressure gradient eliminated by the same projection mechanism that enforces $\nabla \cdot \mathbf{v} = 0$) is

$$\mathbf{R}_v = -(\mathbf{v} \cdot \nabla)\mathbf{v} + \frac{1}{\rho}(\mathbf{J} \times \mathbf{B}). \quad (6)$$

The structural identity between Eqs. (5) and (6) — both are projected by the *same* tensor $\mathcal{P}(\mathbf{k})$, once enforcing the Coulomb gauge and once enforcing incompressibility — is a direct consequence of the fact that both constraints are statements that a vector field’s Fourier coefficients be transverse to $\mathbf{k}$ (Sec. 2.1 of Desigan, 2026).

Dealiasing. Each nonlinear product (the cross products $\mathbf{v} \times \mathbf{B}$, $\mathbf{J} \times \mathbf{B}$, and the advection term $(\mathbf{v} \cdot \nabla)\mathbf{v}$) is computed in real space, transformed to Fourier space, and then masked by the Orszag 2/3 dealiasing mask (Milestone 1) before being projected. This follows Sec. 2’s discussion: a quadratic nonlinearity convolves modes k_1, k_2 into $k_1 + k_2$, which can exceed $k_{\max}$ and alias back into the resolved spectrum unless removed.

5.2 Implementation

`python/hopfnet/rhs.py` implements:

- `spectral_derivative(grid, field_hat, axis)`: $\partial_{x_{\text{axis}}} \rightarrow ik_{\text{axis}}$, used for the nine partial derivatives $\partial v_i / \partial x_j$ in the advection term;
- `compute_B_and_J(grid, A_hat)`: two applications of `spectral_curl` (Milestone 2) give $\hat{\mathbf{B}} = \nabla \times \hat{\mathbf{A}}$ and $\hat{\mathbf{J}} = \nabla \times \hat{\mathbf{B}}$;
- `compute_advection(grid, fft, v_hat)`: computes $(\mathbf{v} \cdot \nabla)\mathbf{v}$ by transforming $\mathbf{v}$ and each $\partial v_i / \partial x_j$ to real space, forming the real-space sum $\sum_j v_j \partial v_i / \partial x_j$, transforming back, and applying the dealias mask;
- `cross_real(a, b)`: elementwise real-space cross product, used for both $\mathbf{v} \times \mathbf{B}$ and $\mathbf{J} \times \mathbf{B}$;
- `compute_R_A` and `compute_R_v`: assemble Eqs. (5) and (6) respectively, each dealiasing every nonlinear product before the final call to `project_field`.

This module is implemented in Python, but every floating-point-intensive step — the FFTs, the curls, and the projection — is a call into the compiled `hopfnet_cpp` extension; the Python layer performs only NumPy array arithmetic on already-transformed buffers.

5.3 Validation

Six unit tests in `tests/test_milestone4.py`:

1. **Spectral derivative:** $\partial_x \cos(x) = -\sin(x)$ reproduced to 10^{-12} via `spectral_derivative` followed by inverse FFT.
2. **Dealiasing mechanism, demonstrated end-to-end:** the product $\cos(3x)\cos(2x) = \frac{1}{2}[\cos(5x) + \cos(x)]$ on an $N = 8$ grid generates an aliased mode at integer wavenumber $n = -3$ (from $5 \rightarrow 5 - 8$), which lies outside the 2/3 cutoff ($\lfloor (2/3) \cdot 4 \rfloor = 2$). Before masking this mode is populated ($|\hat{f}| > 10^{-6}$); after masking it is *exactly* zero, while the surviving mode ($n = 1$) is untouched.
3. **Advection:** for $\mathbf{v} = (\sin x, 0, 0)$, $(\mathbf{v} \cdot \nabla)\mathbf{v} = (\frac{1}{2} \sin 2x, 0, 0)$, reproduced to 10^{-12} .
4. **Lorentz force chain:** for $\mathbf{A} = (0, 0, \sin x)$ (giving $\mathbf{B} = (0, -\cos x, 0)$ and, by a second curl, $\mathbf{J} = (0, 0, \sin x)$), the cross product $\mathbf{J} \times \mathbf{B} = (\frac{1}{2} \sin 2x, 0, 0)$ is reproduced to 10^{-12} , validating the full curl $\rightarrow$ curl $\rightarrow$ cross-product chain.
5. **$\mathbf{R}_A$ gauge consistency:** for random divergence-free $\hat{\mathbf{A}}, \hat{\mathbf{v}}$, $\mathbf{k} \cdot \hat{\mathbf{R}}_A = 0$ to 10^{-10} .
6. **$\mathbf{R}_v$ incompressibility consistency:** likewise, $\mathbf{k} \cdot \hat{\mathbf{R}}_v = 0$ to 10^{-10} .

All six tests pass.

Milestone	Component	Tests
1	Spectral grid, FFT, dealias mask	6
2	Coulomb gauge projection, spectral curl	5
3	Hopf link Biot–Savart initialization	4
4	Nonlinear RHS pipeline ($\mathbf{R}_A$, $\mathbf{R}_v$)	6
Total		21

6 Summary of Test Coverage

All 21 tests pass on $N = 8$ test grids, exercising every operator that will appear in the 128^3 production loop: FFT round-trip and normalization, the Coulomb-gauge projection tensor and its idempotence and $\mathbf{k} = 0$ behavior, the spectral curl and its automatic solenoidality, the regularized Biot–Savart initial condition and its convergence and gauge consistency, and the fully dealiased, doubly-projected nonlinear RHS for both the magnetic and velocity fields.

7 Current Status and Next Steps

7.1 What Is Validated

- All spectral differential operators (∇ , $\nabla \times$, $\nabla \cdot$ via projection) are implemented correctly to machine precision.
- The Coulomb-gauge projection $\mathcal{P}(\mathbf{k})$ correctly enforces $\nabla \cdot \mathbf{A} = 0$ and, by the same construction, $\nabla \cdot \mathbf{v} = 0$ for the incompressible velocity field — including the $\mathbf{k} = 0$ edge case.
- The Hopf link initial condition is generated, regularized, and gauge-projected, and carries genuine (non-trivial, solenoidal) magnetic energy.
- The full nonlinear forcing for both evolved fields, $\mathbf{R}_A$ and $\mathbf{R}_v$, is implemented with Orszag 2/3 dealiasing on every quadratic product, and both satisfy their respective transversality constraints after projection.

7.2 What Remains for Phase 1

- **Milestone 5:** precompute the Kassam–Trefethen ETDRK4 contour-integral coefficients $f_1, f_2, f_3, e_1 = e^{\mathcal{L}\Delta t/2}, e_2 = e^{\mathcal{L}\Delta t}$ for *two* independent linear operators — $\mathcal{L}_A = -\eta k^2 - \eta_4 k^4$ for the induction equation and $\mathcal{L}_v = -\nu k^2 - \nu_4 k^4$ for the momentum equation — using the $M = 32$, $R = 1/\Delta t$ contour described in Sec. 4.3–4.4 of Desigan, 2026.
- **Milestone 6:** assemble the ETDRK4 time-stepping loop coupling $\mathbf{R}_A$ and $\mathbf{R}_v$ from Milestone 4 with the two coefficient sets from Milestone 5, including checkpointing and online diagnostics ($\nabla \cdot \mathbf{A}$, $\nabla \cdot \mathbf{v}$, energy, H_{gen}).
- **Milestone 7:** end-to-end short integration from the Hopf link IC at small grid size, as a fast regression smoke test.
- **128^3 production run:** once Milestones 5–7 pass, execute the full-resolution Hopf link evolution and verify the Algorithmic Validation Strategy of Sec. 6 of Desigan, 2026 (divergence-free to 10^{-13} , $\mathcal{O}(\Delta t^4)$ temporal convergence, H_{gen} conservation under $\eta_4 = d_i = 0$, and a clean $k^{-5/3}$ inertial range with no pile-up at k_{max}).

References

- Desigan, M. R. (2026). *Mathematical Foundations of the HopfNet-MHD Engine*. Phase 1.0 Documentation.
- Kassam, A. K., & Trefethen, L. N. (2005). Fourth-order time-stepping for stiff PDEs. *SIAM Journal on Scientific Computing*, 26(4), 1214-1233.
- Abdelhamid, H. M., Kawazura, Y., & Yoshida, Z. (2015). Hamiltonian formalism of extended magnetohydrodynamics. *Journal of Physics A: Mathematical and Theoretical*, 48(23), 235502.
- Orszag, S. A. (1971). On the elimination of aliasing in finite-difference schemes by filtering high-wavenumber components. *Journal of the Atmospheric Sciences*, 28(6), 1074.